Assignment 03: Page Tables (pgtbl)

CSci 530: Operating Systems

Objectives

In this lab you will explore page tables and modify them to implement common OS features. Before performing this lab, read Chapter 03 of our open source xv6 textbook, and familiarize yourself with page table creation in the following files:

- `kernel/memlayout.h` which captures the layout of memory.
- `kernel/vm.c` which contains most virtual memory (VM) code.
- `kernel/kalloc.c` which contains code for allocating and freeing physical memory.

It may also help to consult the RISC-V privileged architecture manual.

Description

Have a look at the Getting Started instructions for information on how to set up your computer to work on and run these assignments.

Note: Remember that assignments are submitted to GitHub classrooms by creating a commit and pushing it back to your assignment repository. You are required to make a commit for at least each task as you complete it (though you can make more than one commit for a task if needed). In general, from the command line, you can create and push a commit by doing something like:

```
$ git commit -am "My solution for Assignment 03 Task 01, inspect user process page table"
$ git push
```

Assignment Tasks

Task 01: Inspect a user process page table (easy)

To help you understand RISC-V page tables, your first task is to explain the page table for a user process.

Run make qemu and run the user program `pgtbltest`. The `print_pgtbl()` functions prints out the page-table entries for the first 10 and last 10 pages of the `pgtbltest` process using the `pgpte` system call that we added to xv6 for this lab. The output looks as follows:

```
print_pgtbl starting
va 0x0 pte 0x21FCD45B pa 0x87F35000 perm 0x5B
va 0x1000 pte 0x21FD205B pa 0x87F48000 perm 0x5B
...
va 0x3FFFFFE000 pte 0x21FC94C7 pa 0x87F25000 perm 0xC7
va 0x3FFFFFF000 pte 0x2000184B pa 0x80006000 perm 0x4B
```

For every page table entry in the `print_pgtbl()` output, explain what it logically contains and what its permission bits are. Figure 3.4 in the xv6 book might be helpful, although note that the figure might have a slightly different set of pages than the process that's being inspected here. Note that xv6 doesn't place the virtual pages consecutively in physical memory. Create a file named `answers-pgtbl.txt` in the top level of this repository with your answer, and add and commit it for your task 01 submission.

Hint: When looking at figure 3.4 and the output from `print_pgtbl()` you should be able to indentify pretty much one of each of the different types of pages illustrated in the process's user address space.

Note: The `print_pgtbl` will run correctly and output `OK` at this point, but it is expected and correct that the other things that run will not succeed yet and cause errors on the output after the page table is displayed.

Task 02: Print a page table (easy)

To help you visualize RISC-V page tables, and perhaps to aid future debugging, your next task is to write a function that prints the contents of a page table.

We added a system call `kpgtbl()` which calls `vmprint()` in vm.c. It takes a `pagetable_t` argument (the page table of the process that made the system call). Your job is to print that pagetable in the format described below.

When you run `pgtbltest`, the `print_kpgtbl()` test calls your `vmprint()` which should print the following:

The first line displays the argument to `vmprint()`. After that there is a line for each PTE, including PTEs that refer to page-table pages deeper in the tree. Each PTE line is indented by a number of " .." that indicates its depth in the tree. Each PTE line shows its virtual addresss, the pte bits, the physical address extracted from the PTE, and the permission bits in the PTE: R(ead), W(rite), eXecute, and U(ser). Don't print PTEs that are not valid. In the above example, the top-level page-table page has mappings for entries 0 and 255. The next level down for entry 0 has only index 0 mapped, and the bottom-level for that index 0 has a few entries mapped.

Your code might emit different physical addresses than those shown above. The number of entries and the virtual addresses should be the same.

Some **Hints**:

- Use the macros at the end of the file `kernel/riscv.h`
- The function `freewalk()` may be inspirational. You may want to add in a new function that performs the actual recursive calls, that will be invoked from the `vmprint()` function. For example maybe something called `vmwalk()` that might take additional parameters like the tree level and/or the virtual address base constructed so far.
- Use `%p` in your `printf()` calls to print out full 64-bit hex PTEs and addresses as shown in the exmaple. Page table entries (`pte`) and using the macro to display the physical address of a page table entry are both `uint64` types. You can use a `(void *)` cast to cast these to appropriate type expected by the `%p` formatter.
  - You will need to construct the virtual address here. The index into the page directory tells you a set of 9 bits in the virtual address being looked up. You should use an `uint64` type and some bit shifting and manipulation to construct the virtual address being printed in this function.

For every leaf page in the `vmprint()` output, explain what it logically contains and what the permission bits allow and disallow, and how it relates to the output of the earlier `print_pgtbl()` exercise above. Again, figure 3.4 in the xv6 book might be helpful. Give your written answers in `answers-pgtbl.txt` again, below your answers for the first question task.

Task 03: Speed up the `getpid` system call (medium)

Some operating systems (e.g., Linux) speed up certain system calls by sharing data in a read-only region between userspace and the kernel. This eliminates the need for traps and transition from user mode into kernel mode when performing these system calls, which can be expensive in terms of operations needed to perform the user to kernel mode switch. To help you learn how to insert mappings into a page table, your task is to implement this optimization for the getpid() system call in xv6.

When each process is created, map one read-only page at `USYSCALL` (a virtual address defined in `memlayout.h`). At the start of this page, store a struct `usyscall` (also defined in `memlayout.h`), and initialize it to store the `PID` of the current process. For this lab, `ugetpid()` has been provided on the userspace side and will automatically use the `USYSCALL` mapping. You will receive full credit for this part of the lab if the ugetpid test case passes when running pgtbltest.

Some **hints**:

- Choose permission bits that allow userspace to only read the page.
- There are a few things that need to be done over the lifecycle of a new page. For inspiration, understand the trapframe handling in `kernel/proc.c`
  - Don't forget to add a struct to `kernel/proc.h` as well
  - The only difference from allocating and deallocating the trapframe for the process and the usyscall page is that you will need to initialize the pid value on this page when a fork occurs.

Which other xv6 system call(s) could be made faster using this shared page? Explain how in the `answers-pgtbl.txt` file for this part of the assignment.

Task 04: Use superpages (hard)

The RISC-V paging hardware supports two-megabyte pages as well as ordinary 4096-byte pages. The general idea of larger pages is called superpages, and (since RISC-V supports more than one size) 2M pages are called megapages. The operating system creates a superpage by setting the `PTE_V` and `PTE_R` bits in the level-1 PTE, and setting the physical page number to point to the start of a two-megabyte region of physical memory. This physical address must be two-mega-byte aligned (i.e., a multiple of two megabytes). You can read about this in the RISC-V privileged manual by searching for megapage and superpage; in particular, the top of page 136. Use of superpages decreases the amount of physical memory used by the page table, and can decrease misses in the TLB cache. For some programs this leads to large increases in performance.

Your job is to modify the xv6 kernel to use superpages. In particular, if a user program calls `sbrk()` with a size of 2 megabytes or more, and the newly created address range includes one or more areas that are two-megabyte-aligned and at least two megabytes in size, the kernel should use a single superpage (instead of hundreds of ordinary pages). You will receive full credit for this part of the lab if the `superpg_fork()` and `superpg_free()` test cases pass when running `pgtbltest`.

Some **hints**:

- Read `superpg_fork()` and `superpg_free()` in `user/pgtbltest.c` For example, `superpg_fork()` first uses `sbrk(SZ)` to attempt to allocate `8 * SUPERPGSIZE` amount of memory. Immeidatalty after, the `supercheck()` function is called. This is the function that is performing tests to see that you allocated super pages. For example in the second `for` loop, it is walking some virtual addresses on the last (super) page that was allocated. If super pages are used, it is expected that all of the page table entries will be the same because there is only 1 super page for the whole range. But if super pages are not successfully used, then different page table entries will be found, and this causes the check to report an error.
- (easy) Your kernel will need to be able to allocate and free two-megabyte regions. Modify `kalloc.c` to set aside a few two-megabyte areas of physical memory, and create `salloc()` and `sfree()` functions (which stand for super page allocator and super page free respectively). You'll only need a handful of two-megabyte chunks of memory. I will give some partial credit on task 04 if you have your 2M allocation working, but don't get all of the superpages implemented and working completely.
  Modifying this code is helpful in understanding how the kernel keeps track of unallocated memory space using linked lists, and allows for dynamic allocation of memory.
  - As a suggestion, you might find it useful to define the following in `memlayout.h` for use in setting up two separate linked lists of free pages to be managed in the `kalloc.c` routines:

```
#define KERNBASE     0x80000000L
#define SUPERPGSTART (KERNBASE + 90 * 1024 * 1024)
#define PHYSTOP      (SUPERPGSTART + 38 * 1024 * 1024)
```

- A good place to start is `sys_sbrk` in `kernel/sysproc.c`, which is invoked by the `sbrk` system call. Follow the code path to the function that allocates memory for sbrk.
- You will need to touch many of the functions in `kernel/vm.c` to successfully complete this task, including probably all of `uvmalloc()`, `uvmunmap()`, `uvmcopy()`, `mappages()`, `walk()`
  - As a big hint, a superpage uses 21 bits of offset instead of the 12 for a regular 4K page. This means superpages only have L2 and L1 page table entries, and any L1 page table directory entry where `PTE_R` and `PTE_V` are set will be treated as a mapping to a superpage. This means that `walk()` has to handle sometimes only going down to level 1, but other times to level 0. An alternative approach might be to make an `swalk()` function that is called appropriately when walking to allocate superpages at appropriate L1 entries.

  - Note that the following declarations were already given for you in `kernel/riscv.h`, which you will need to use:

```
#define SUPERPGSIZE          (2 * (1 << 20)) // bytes per page
#define SUPERPGROUNDUP(sz)   (((sz) + SUPERPGSIZE - 1) & ~(SUPERPGSIZE - 1))
#define SUPERPGROUNDDOWN(sz) (SUPERPGROUNDUP(sz) - SUPERPGSIZE)
```

  - Likewise the `PTA2PA` AND `PA2PTA` macros given in `kernel/riscv.h` won't work if you are translating a virtual address to physical address or the other way for a superpage in the L1 of the page table tree. The superpages use 21 bits for a page offset / page shift.
- Superpages must be allocated when a process with superpages forks, and freed when it exits; you'll need to modify `uvmcopy()` and `uvmunmap()`.

Real operating systems dynamically promote a collection of pages to a superpage. The following reference explains why that is a good idea and what is hard in a more serious design: Juan Navarro, Sitaram Iyer, Peter Druschel, and Alan Cox. Practical, transparent operating system support for superpages. SIGOPS Oper. Syst. Rev., 36(SI):89-104, December 2002. This reference summarizes superpage-implementations for different OSes: A comprehensive analysis of superpage management mechanism and policies.

Submit the Assignment

Time spent

Create a new file, `time.txt`, and put in a single integer, the number of hours you spent on the lab. Make sure you add and commit a final commit with this file as it is part of the autograder score.

Answers

This lab had questions asked in Task 01. Make sure you have a file named `answer-pgtbl.txt` with the answer for all questions asked in tasks 1-4 in this file added and pushed as a commit. This file is checked as part of the final autograder score as well.

Assignment submissions are handled by the GitHub classroom autograder. You should create at least 1 commit for each task as you do it, and push it to your GitHub classroom. You can check that the autograder tests are passing in GitHub classroom by looking at your submitted releases. The tests run in the GitHub classroom are the same ones that you can run locally by using `make grade` or `./grade-lab-pgtbl test`.

- Please run `make grade` to ensure that your code passes all of the tests. The GitHub classroom autograder will use the same grading program to assign you an autograder evaluation score for your work.

Optional Challenge Exercises

- Implement some ideas from the paper referenced above to make your super-page design more real.
- Unmap the first page of a user process so that dereferencing a null pointer will result in a fault. You will have to change user.ld to start the user text segment at, for example, 4096, instead of 0.
- Add a system call that reports dirty pages (modified pages) using PTE_D.